

06026240 INTELLIGENT SYSTEM DEVELOPMENT

Ch X : API

with Dr. Bhattarabhorn Wattanacheep

1. API คืออะไร และทำงานอย่างไร

- นิยาม ประเภทของ API และภาพรวมระบบ Frontend → API → Backend → AI Model

2. ออกแบบ RESTful API

- CRUD, Endpoint และ path, การกรองและเรียงข้อมูล, HTTP status code

3. คุณลักษณะของ API

- Versioning, เอกสาร API, Security, การจัดการข้อผิดพลาด, การทดสอบ

4. Hands-on: สร้าง API ด้วย FastAPI

- เตรียม environment → รัน FastAPI → เชื่อม MySQL (XAMPP) → แยก config ลง .env → ต่อหน้าเว็บ

5. ส่วนขยายและการนำขึ้นใช้งาน

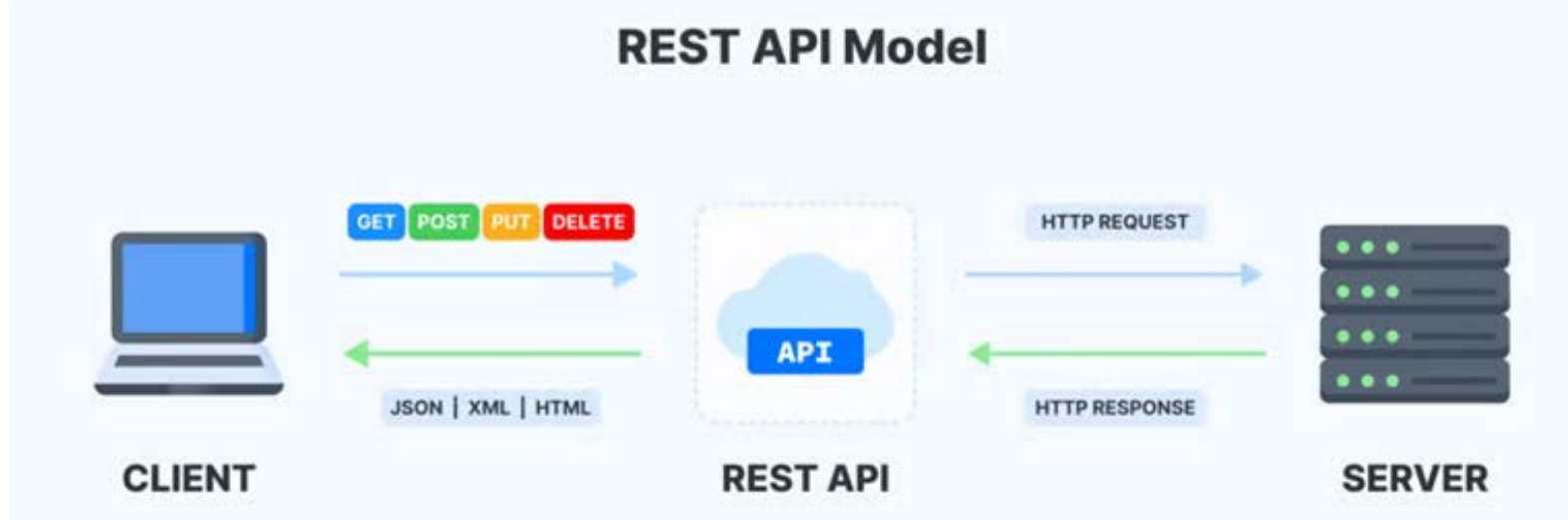
- เรียกโมเดล AI ผ่าน API, เลือกระหว่างรัน Local กับ Docker, ลำดับการ deploy

ความสามารถที่ได้

- อธิบายได้ว่า Client ส่ง Request และ Server ตอบ Response อย่างไร
- ออกแบบ Endpoint และเลือก HTTP method ให้ตรงกับงาน CRUD
- อ่าน HTTP status code เพื่อบอกได้ว่า request ผ่าน ล้มเหลว หรือเขียนผิด
- สร้าง FastAPI ที่เชื่อมกับฐานข้อมูล MySQL ได้ด้วยตัวเอง
- แยกรหัสผ่านและ config ออกจากโค้ดด้วยไฟล์ .env
- ต่อหน้าเว็บ index.html เข้ากับ API ของตัวเอง

สิ่งที่ต้องเตรียมก่อนเริ่มแลป

- Python 3.11+ (ติ๊ก Add to PATH), VS Code, XAMPP, และ Postman หรือหน้า /docs ของ FastAPI

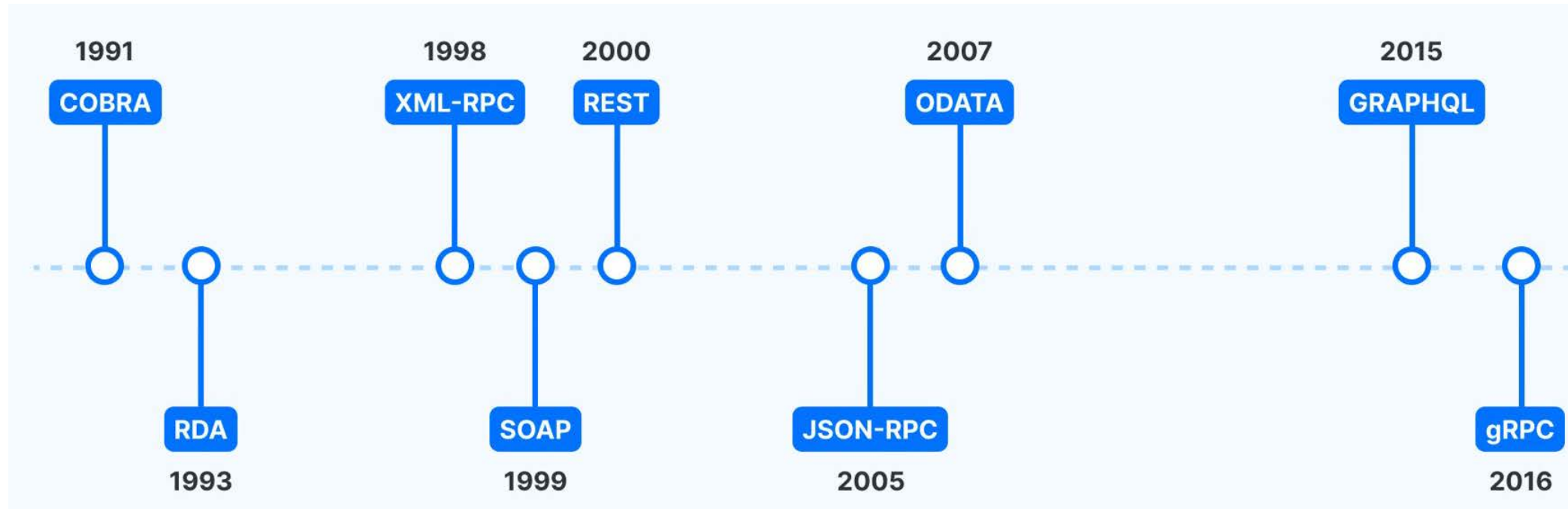


API (Application Programming Interface) คือ ตัวกลางเชื่อมต่อระหว่างแต่ละอุปกรณ์ซึ่งอนุญาตให้แลกเปลี่ยนข้อมูลระหว่าง Server กับ Client โดย Server รับ Requests จาก Client และส่ง Response ข้อมูลกลับ

เป้าหมาย:

- อำนวยความสะดวกในการสื่อสารระหว่างระบบต่าง ๆ
- ให้บริการฟังก์ชันและข้อมูลจากระบบหนึ่งไปยังอีกระบบหนึ่ง
- ลดจำนวนโค้ดซ้ำๆ ที่จำเป็นในการสร้างแอป

API Timeline



RESTful API:

- Web API (REST, SOAP, GraphQL)
- Data Format (JSON, XML)

โจทย์เดียวกัน: ขอชื่อและอีเมลของผู้ใช้ id 1 — สามรูปแบบใช้ HTTP เหมือนกันแต่เขียนคำขอต่างกัน

REST

ระบุทรัพยากรด้วย URL และใช้ HTTP method

`GET /users/1`

`Accept: application/json`

ฟิลด์ที่ได้ = ตามที่เซิร์ฟเวอร์กำหนดไว้

SOAP

ส่ง XML envelope ไปที่ endpoint เดียว

`POST /UserService`

`<soap:Envelope>`

`<GetUser>`

`<Id>1</Id>`

`</GetUser>`

`</soap:Envelope>`

GraphQL

ระบุฟิลด์ที่ต้องการในตัว query

`POST /graphql`

`query {`

`user(id: 1) {`

`name email`

`}`

`}`

JSON

ค่าเริ่มต้นของงานปัจจุบัน อ่านง่ายและขนาดเล็ก

`{"id": 1, "name": "Ann", "email": "ann@mail.com"}`

XML

ยังพบในระบบเดิมและ SOAP

`<user>`

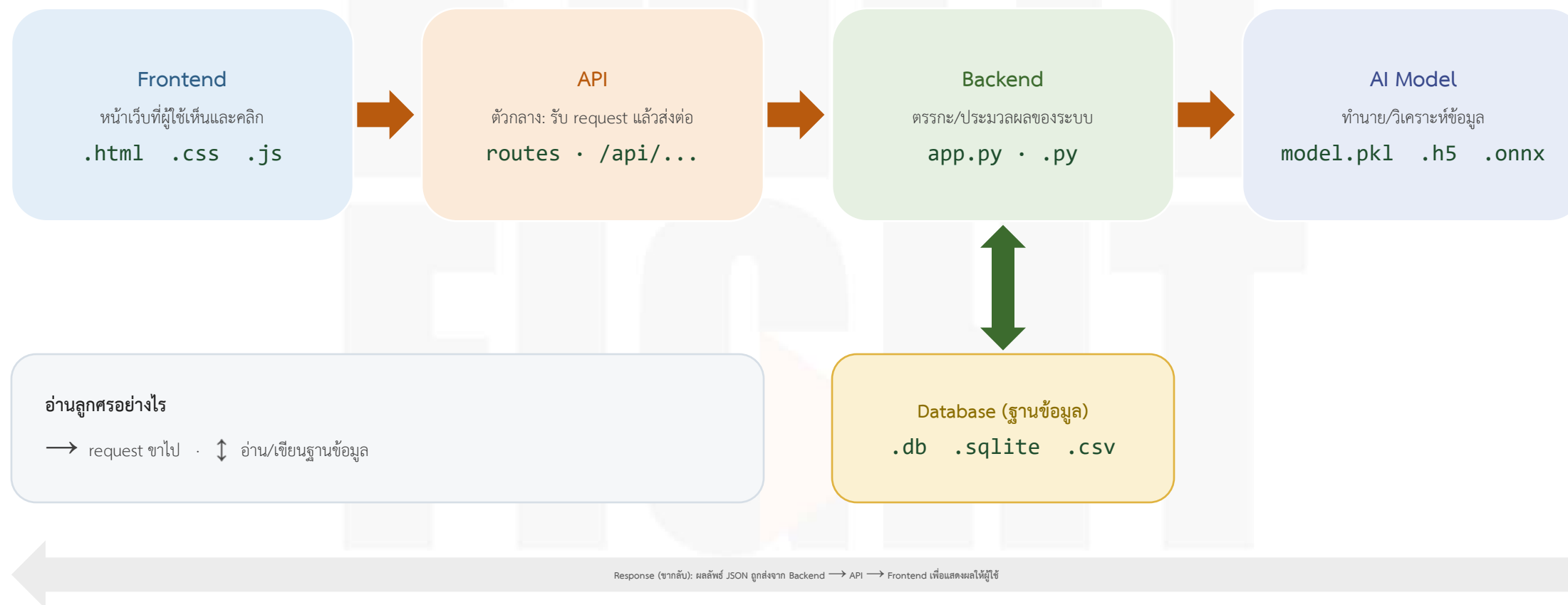
`<id>1</id>`

`<name>Ann</name>`

`</user>`

ภาพรวมระบบ: Frontend → API → Backend → AI Model

ผู้ใช้คลิกบนหน้าเว็บ → คำขอวิ่งผ่าน API → Backend ประมวลผลและเรียก AI Model → ส่งผลลัพธ์กลับมาแสดงผล



ส่วนนี้ตอบ 4 คำถามของการออกแบบ API

- API ทำอะไรได้บ้าง → CRUD กับ HTTP method (GET, POST, PUT, DELETE)
- เรียกที่ไหน → โครงสร้าง Endpoint และการตั้งชื่อ path
- ขอข้อมูลเฉพาะที่ต้องการอย่างไร → Filtering และ Sorting
- รู้ผลลัพธ์ได้อย่างไร → HTTP status code และการจัดการข้อผิดพลาด

ทำไมต้องออกแบบก่อนเขียนโค้ด

- เมื่อเผยแพร่ API ออกไปแล้ว การแก้ path หรือ method ภายหลังทำให้ผู้ใช้เดิมพัง

RESTful API:

- ใช้ HTTP methods (GET, POST, PUT, DELETE)

OPERATION	SQL	HTTP
Create	INSERT	PUT/POST
Read	SELECT	GET
Update	UPDATE	PUT/PATCH
Delete	DELETE	DELETE

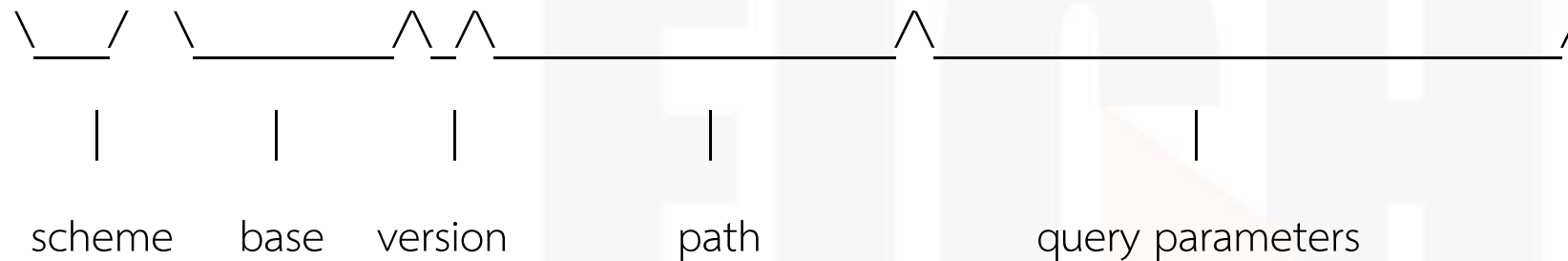
Image Source: railsbridge

ใช้ HTTP methods (GET, POST, PUT, DELETE)

ใช้ URL สำหรับระบุทรัพยากร

Endpoint :

`https://api.abc.com/v1/companies/123/users/?attributes=first_name,last_name`



ตัวอย่าง การกำหนด path

พนักงานหลายคน:

GET <https://api.abc.com/v1/employees>

ใช้ Plural หรือ Singular nouns (จะเลือกใช้อะไรก็ตาม ก็ควรให้มันสอดคล้องกับ APIs ทั้งหมดที่มี)

แบบการกรอง:

GET <https://api.abc.com/v1/employees?year=2011&sort=desc>

GET <https://api.abc.com/v1/employees?section=economy&year=2011>

ตัวอย่าง:

?last_name=Johnson&date_of_birth=1999-12-31

จะ Filter ข้อมูล ด้วย attribute last_name ที่เท่ากับ Johnson และ date_of_birth เท่ากับ 1999-12-31

ใช้สัญลักษณ์เท่ากับ (=) สำหรับเทคนิคที่ใช้เปรียบเทียบข้อมูลที่เท่ากัน และสำหรับการ

Advanced Filtering

ตัวอย่าง:

?filters=creation_date >= 2001-09-20T13:00:00 and creation_date <= 2001-09-21T13:00:00 and
post_code = 3000

จะแสดง collection ของ resources เมื่อ creation_date อยู่ระหว่าง 2001-09-20 1pm และ 2001-09-21 1pm และ
post_code คือ 3000

Sorting

การที่ข้อมูลสามารถกำหนดตามลำดับการเรียงที่เฉพาะเจาะจงได้มักเป็นความต้องการจากแอปพลิเคชันไคลเอนต์ ดังนั้นจึงเป็นเรื่องสำคัญที่จะต้องให้ความยืดหยุ่นแก่ผู้ใช้ในการดึงข้อมูลตามลำดับที่พวกเขาต้องการ พารามิเตอร์สองตัวที่ใช้ในการเรียงลำดับมีดังนี้:

Query Parameter	Description
sort	ในการจัดเรียงข้อมูลเช่น asc หรือ desc
sort_fields	พารามิเตอร์ที่ใช้จัดเรียงเช่น id หรือ name

ตัวอย่าง:

?sort=asc&sort_fields=name,last_modified

?sort=desc&sort_fields=name&sort_fields=last_modified

คำค้นหาทั้งสองนี้ควรเรียงลำดับตามชื่อจากนั้นตามวันที่แก้ไขล่าสุด

HTTP response status code (รหัสสถานะ)

เมื่อ client สร้าง request ไปยัง server ผ่าน API, ควรจะรู้ผลสะท้อนกลับ ไม่ว่าการ request นั้น จะ Pass, Fail หรือ การ request นั้นผิด โดยที่ HTTP status จะเป็นรหัสมาตรฐานที่บอกสถานะของการ request ซึ่งจะมีหลากหลายในแต่ละสถานการณ์ ซึ่ง ตัว Server เองควรจะ return status code ให้ถูกต้อง

ต่อไปนี้จะเห็น HTTP status code ที่สำคัญโดยแบ่งออกเป็นหมวดดังนี้:



GET		POST		PUT		DELETE		PATCH		All	
Code	Status	Code	Status	Code	Status	Code	Status	Code	Status	Code	Status
200	OK	201	Created	202	Accepted	202	Accepted	202	Accepted	408	Request Timeout
400	Bad Request	202	Accepted	204	No content	204	No content	204	No content	501	Method Not Implemented
401	Unauthorised	400	Bad Request	400	Bad Request	400	Bad Request	400	Bad Request		
403	Forbidden	401	Unauthorised	401	Unauthorised	401	Unauthorised	401	Unauthorised		
404	Not found	403	Forbidden	403	Forbidden	403	Forbidden	403	Forbidden		
405	Not Allowed	404	Not found	404	Not found	404	Not found	404	Not found		
415	Unsupported Media Type	405	Not Allowed	405	Not Allowed	405	Not Allowed	405	Not Allowed		
500	Internal Server error	415	Unsupported Media Type	415	Unsupported Media Type	415	Unsupported Media Type	415	Unsupported Media Type		
		422	Unprocessable Entity	422	Unprocessable Entity	500	Internal Server error	422	Unprocessable Entity		
		500	Internal Server error	500	Internal Server error			500	Internal Server error		

- Error Codes:

- 400 Bad Request: คำขอที่ไม่ถูกต้อง
- 401 Unauthorized: การรับรองความถูกต้องล้มเหลว
- 403 Forbidden: ไม่มีสิทธิ์เข้าถึง
- 404 Not Found: ไม่พบทรัพยากร
- 500 Internal Server Error: ข้อผิดพลาดที่เซิร์ฟเวอร์

- Error Messages:

- ให้ข้อมูลที่ชัดเจนเกี่ยวกับปัญหาที่เกิดขึ้น

```
{  
  "status": 400,  
  "error": "RegistrationFailed",  
  "message": "Prerequisite course not completed"  
}
```

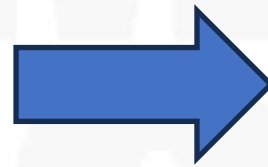
คือส่วนที่บอกว่า APIs มีรุ่นและการเปลี่ยนแปลงอย่างไร และปัจจุบันเป็นรุ่นอะไร เมื่อคุณเผยแพร่ API เป็น Public แล้วตั้งนั้น API ของคุณจะต้องไม่เกิดการ Breaking (อยู่ดี ๆ ใช้งานไม่ได้)

ตัวอย่าง ตอนแรกมี API นี้อยู่

`https://api.abc.com/articles`

//Response result

```
[  
  {  
    "title": "...",  
    "text": "..."  
  }  
  ...  
]
```



API ใหม่แต่แก้ไขใหม่

`https://api.abc.com/articles`

//Response result

```
{  
  "data": [  
    {  
      "title": "...",  
      "detail": "..."  
    }  
    ...  
  ]  
}
```

คำแนะนำการออกแบบ API Versions

โดย API version ควรจะ กำหนดไว้อยู่เสมอ (เช่น ค่าเริ่มเป็นเวอร์ชันล่าสุด)

โดย API version ควรจะ ระบุด้วย 'v'

ถ้าผู้ใช้งานพยายามเข้าถึง API ด้วยการไม่ระบุเวอร์ชัน ดังนั้นเขาก็ควรได้รับข้อความผิดพลาดตอบกลับ 404

ถ้าผู้ใช้งานพยายามเข้าถึง API ด้วยการไม่ระบุ resource (เช่น GET .../api/v1) การตอบกลับอาจจะประกอบด้วย ข้อมูลพื้นฐานเกี่ยวกับ API (เช่น ชื่อ API, และรายละเอียดอื่น ๆ)

โดยอาจจะมี API version หลายอันได้ตัวอย่าง:

.../api/v1/articles

.../api/v2/articles

```
{  
  "name": "Articles API",  
  "version": "v1",  
  "resources": [  
    "/articles",  
    "/users"  
  ]  
}
```

รายละเอียดของ APIs Version เป็นไปตาม semantic versioning:

{MAJOR}.{MINOR}.{PATCH} 2.5.3

เวอร์ชันแรกของ API ต้อง เริ่มต้นด้วยเวอร์ชัน MAJOR เป็น 1

ใช้แนวทางต่อไปนี้เมื่อเพิ่มหมายเลขเวอร์ชัน API:

MAJOR เวอร์ชัน เมื่อทำการ incompatible ของการแก้ไข API

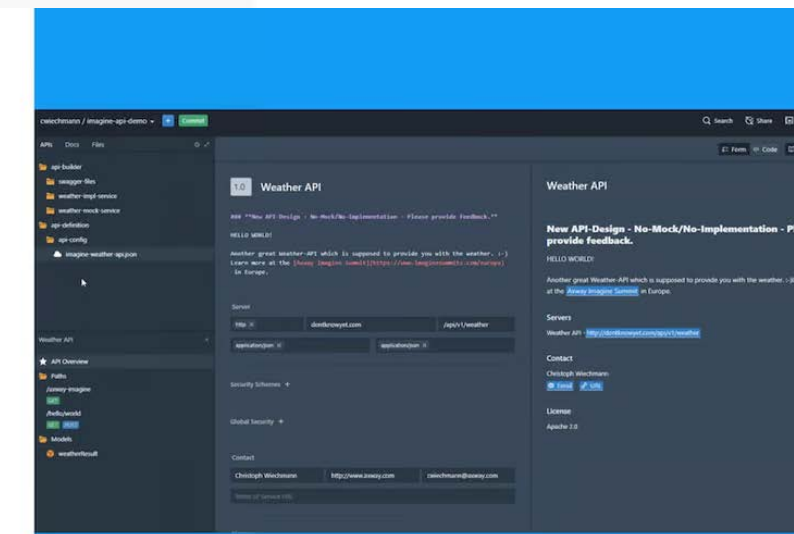
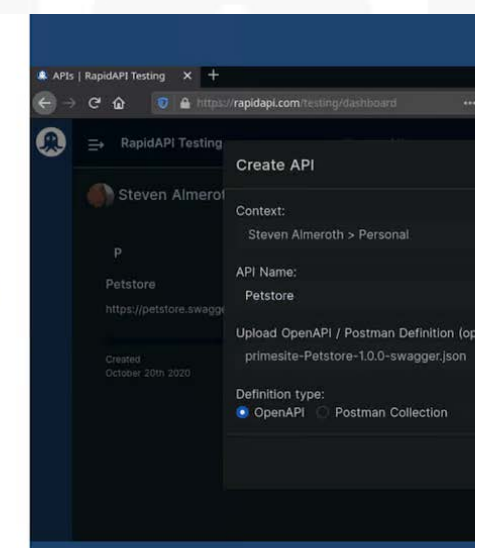
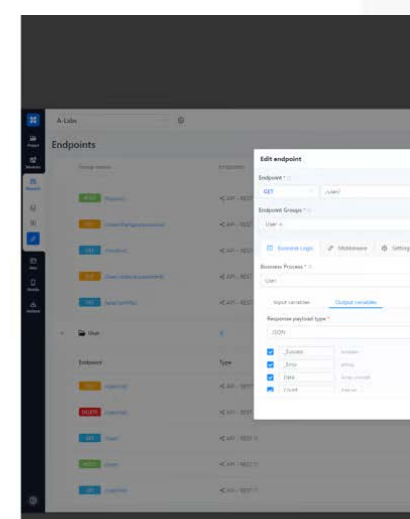
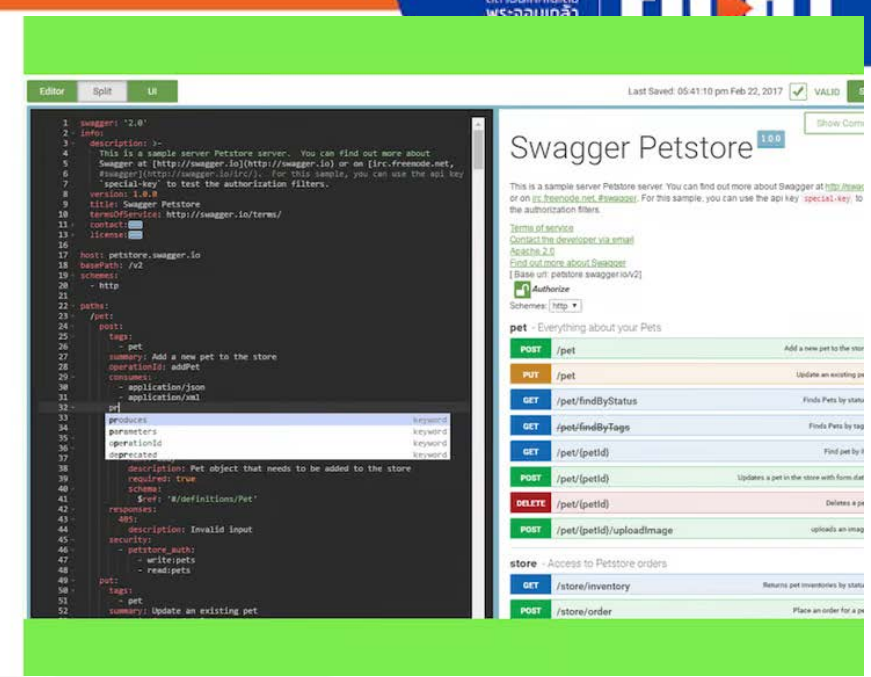
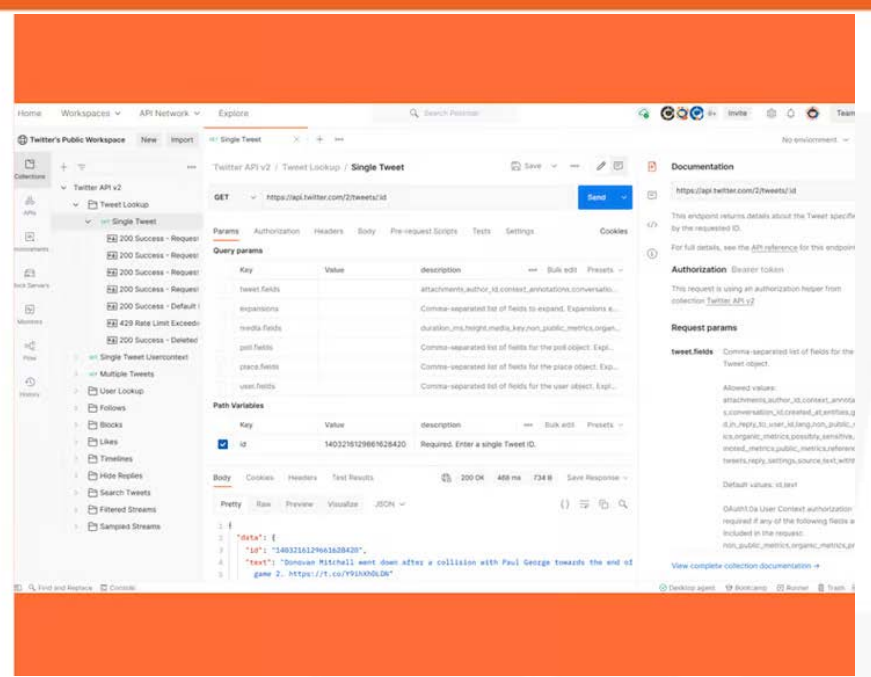
MINOR เวอร์ชัน เมื่อเพิ่มฟังก์ชันใน backwards-compatible หรือคุณสมบัติอื่น ๆ

PATCH เวอร์ชัน เมื่อทำการ backwards-compatible การแก้ไขข้อบกพร่อง

เอกสาร (API Docs) เป็นสิ่งที่สำคัญสำหรับนักพัฒนาในการใช้ API โดย API ที่แตกต่างกันจะมีลักษณะการทำงานที่แตกต่างกัน ซึ่งต้องใช้พารามิเตอร์ที่แตกต่างกัน เช่น วิธีการ HTTPS การ Response data ของ API ดังนั้นนักพัฒนาส่วนใหญ่จะชอบเอกสารที่ดีสำหรับใช้อ้างอิงการใช้งาน API

ตัวอย่างเครื่องมือ เพื่อช่วยนักพัฒนาในการสร้างเอกสาร API มีดังนี้:

เครื่องมือออกแบบ API



Security Features

ความปลอดภัย	เหตุการณ์ของ API	รายละเอียด	ความจำเป็นในการ Implementation
HTTP verbs	Listeners->Path	HTTP verbs ต่าง ๆ สามารถเลือกใช้ได้จาก Path ของ API ที่กำหนดไว้	ควร
Input Validation	Filter -> Content Filtering	สามารถ Filters สำหรับการ Validation ข้อมูล Input ตัวอย่างเช่น การตรวจสอบขนาดของข้อความ การตรวจสอบ Schema การตรวจสอบ Headers หรือการตรวจสอบ Query strings เป็นต้น	ควร
SSL	Listeners	ใช้ SSL protocol (เช่น TLS 1.2)	ควร
Digital Certificate	Filter -> Integrity	สามารถ Filters สำหรับใช้ Signature เพื่อการตรวจสอบตัวอย่างเช่น API Token หรือ JWT sign	เป็น Optional หรือขึ้นอยู่กับ Business requirements
JWT	Filter -> Integrity	ข้อความสามารถเข้ารหัสให้อยู่ในรูปแบบ JWT	เป็น Optional หรือขึ้นอยู่กับ Business requirements

ความปลอดภัย	เหตุการณ์ของ API	รายละเอียด	ความจำเป็นในการ Implementation
API Keys	Filter -> Authentication	สามารถ Filters สำหรับการ Authentication ผู้ใช้งาน เช่น API Keys หรือ API Token	ควร
OAuth	Filter -> OAuth	สามารถใช้ OAuth สำหรับมอบคุณสิทธิ์เข้าถึง Data resources สำหรับผู้ใช้งาน	เป็น Optional หรือขึ้นอยู่กับ Business requirements
CORS	Listeners->Path	CORS สามารถจำกัดการเข้าถึงในระดับ Path level	ควร

อย่าส่งผ่านข้อมูลที่ละเอียดอ่อน (Sensitive data) ใน URL เช่น รหัสผ่าน JWT token หรือ API keys
เนื่องจากข้อมูลนี้อาจจะเก็บไว้ในบันทึกของเบราว์เซอร์และเซิร์ฟเวอร์

ตัวอย่างที่ไม่ควร

`https://api.example.com/namespace/v1/auth/login/username=abcd123456789&password=123321`

URL ด้านบนแสดงรหัสผ่าน ดังนั้นอย่าใช้รูปแบบนี้ในระบบ Authentication

- Manual Testing:
 - ใช้เครื่องมือ เช่น Postman หรือ Insomnia
- Automated Testing:
 - ใช้เครื่องมือเช่น Postman Tests หรือ JMeter
- API Documentation:
 - เพื่อสร้างเอกสาร API ที่สามารถอ่านและทดสอบได้

สองแอปใช้ FastAPI ตัวเดียวกัน ต่างกันที่ชั้นกลางและเรื่องการเก็บข้อมูล

Curriculum App — ถามตอบข้อมูลหลักสูตร

รับคำถามภาษาไทย → Qwen เขียน SQL → รันกับ SQLite → คืนคำตอบที่อ้างอิงข้อมูลจริง

ไฟล์: main.py · database.py · model_service.py · schemas.py · config.py

มีฐานข้อมูล เพราะข้อมูลหลักสูตรเป็นข้อมูลสาธารณะที่ต้องค้นซ้ำ

Transcript App — สกัดข้อมูลจากใบแสดงผลการเรียน

รับไฟล์อัปโหลด → preprocessing → Typhoon OCR → Qwen จัดเป็น JSON → ลบไฟล์ทิ้ง

ไฟล์: main.py · pipeline_service.py · schemas.py · config.py

ไม่มีฐานข้อมูล เพราะเป็นเอกสารส่วนบุคคล จึงไม่เก็บอะไรไว้หลังตอบกลับ

ทั้งสองแอปต่อยอดจากแลปก่อนหน้า Curriculum เรียก Lab 8B ส่วน Transcript เรียก Lab 8A ซึ่งใช้ Lab 7A เป็น OCR engine

เปิดหน้า /docs แล้วจะเห็นรายการนี้ทั้งหมด

GET	/	หน้าเว็บของเรา ส่งไฟล์ index.html กลับไป
GET	/api/health	บอกว่าไฟล์ฐานข้อมูลมีจริงไหม และ Ollama ตอบไหม
GET	/api/program	ข้อมูลหลักสูตร ถ้าไม่มีข้อมูลตอบ 404
GET	/api/courses	ค้นรายวิชา รับ search, limit, offset ทาง URL
POST	/api/courses	เพิ่มรายวิชา สำเร็จตอบ 201 รหัสซ้ำตอบ 409
POST	/api/ask	ถามเป็นภาษาไทย ส่ง body เป็น {"question": "..."}

สามจุดที่อยากให้เห็น

GET รับค่าทาง query parameter ส่วน POST รับทาง JSON body ตรงกับตาราง CRUD ก่อนหน้านี้

ทุก endpoint ประกาศ response_model ไว้ FastAPI จึงตรวจรูปแบบข้อมูลขาออกให้เอง

/api/health คือปุ่มแรกที่เราควรทดสอบ ถ้าตรงนี้ไม่ผ่าน endpoint อื่นจะพังตามทั้งหมด

แอปนี้มีแค่สามเส้นทาง แต่เส้นทางหลักรับไฟล์ ไม่ใช่รับ JSON

GET / หน้าเว็บของเรา ส่งไฟล์ index.html กลับไป

GET /api/health บอกว่าใช้โมเดล OCR ตัวไหน โมเดลข้อความตัวไหน และเพดานขนาดไฟล์

POST /api/transcript/extract อัปโหลดใบแสดงผลการเรียนรู้แล้วได้ JSON กลับ

เส้นทางอัปโหลดส่งแบบ multipart/form-data ไม่ใช่ JSON

file ไฟล์ที่อัปโหลด รองรับ PDF, PNG, JPG, TIFF เท่านั้น

preprocessing วิธีเตรียมภาพก่อนเข้า OCR ค่าเริ่มต้นคือ none

include_markdown ขอผลแบบ markdown แนบมาด้วยหรือไม่ ค่าเริ่มต้นคือไม่

ด้านตรวจที่เขียนไว้ก่อนเริ่มประมวลผล

นามสกุลไฟล์ไม่ถูกต้อง ตอบ 415 · ไฟล์ใหญ่เกินที่ระบบจะรับ ตอบ 413 · OCR อ่านไม่ได้ ตอบ 422

ไฟล์ถูกเขียนลงฟลैชไดรฟ์ชั่วคราวและถูกลบอัตโนมัติเมื่อจบคำขอ ไม่มีสำเนาค้างในเครื่อง

1 . สร้างและเปิดใช้ virtual environment

`python -m venv venv` แล้ว `venv\Scripts\activate` (Windows)

เห็น (venv) หน้าบรรทัดคำสั่งแปลว่าเปิดใช้แล้ว

2 . ติดตั้ง package ของโปรเจกต์

`pip install -r requirements.txt`

3 . เปิด Ollama และโหลดโมเดลที่แอปเรียกใช้

`ollama serve` เปิดค้างไว้อีกหน้าต่างหนึ่ง

`ollama pull` ชื่อโมเดล Qwen สำหรับเขียน SQL และจัด JSON · Typhoon OCR สำหรับอ่านใบเกรด

4 . ตรวจไฟล์ .env ให้ตรงกับเครื่องตัวเอง

`ollama_url · ollama_model · ocr_model · db_path · max_upload_mb`

ไฟล์ฐานข้อมูลของ Curriculum App ต้องมีอยู่จริงตาม `db_path` ไม่งั้นทุกเส้นทางจะตอบ 503

รันทีละแอป คนละพอร์ต จาก terminal ของ VS Code

uvicorn ชื่อแฟ็กเกจ.main:app --reload --port 8000 Curriculum App

uvicorn ชื่อแฟ็กเกจ.main:app --reload --port 8001 Transcript App

รันจากโฟลเดอร์แม่ของแฟ็กเกจเสมอ เพราะโค้ดใช้ import แบบ relative เช่น .config

อยากดู FastAPI บนเว็บ เปิดสามที่นี้

http://127.0.0.1:8000/ หน้าเว็บของเรา ที่มาจากไฟล์ index.html

http://127.0.0.1:8000/docs Swagger UI กดทดสอบได้ทุก endpoint โดยไม่ต้องเขียนโค้ด

http://127.0.0.1:8000/redoc เอกสาร API แบบอ่านอย่างเดียว ใช้ส่งให้คนอื่นดู

ใน terminal กด Ctrl แล้วคลิกที่ลิงก์ เปิดเบราว์เซอร์ได้เลย

สิ่งที่ควรรู้

--reload คือแก้โค้ดแล้วเซิร์ฟเวอร์รีสตาร์ทเอง ใช้ตอนพัฒนาเท่านั้น

พอร์ตชนกันให้เปลี่ยนเลขหลัง --port ปิดเซิร์ฟเวอร์ด้วย Ctrl + C

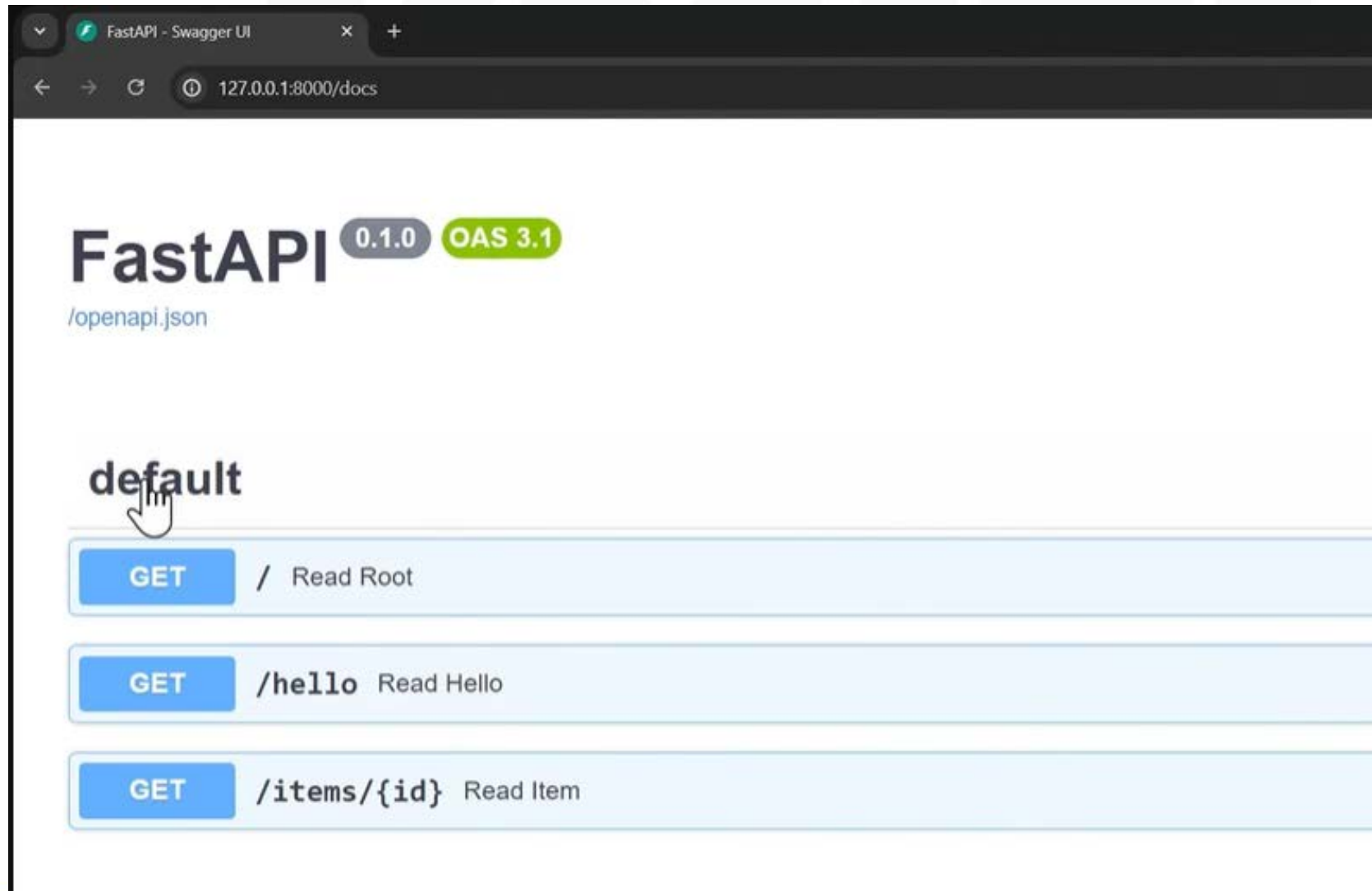
รหัสทั้งหมดนี้เราเขียนไว้เองในโค้ดด้วย HTTPException ไม่ได้เกิดขึ้นเอง

201	เพิ่มรายวิชาสำเร็จ	สร้างข้อมูลใหม่แล้ว
409	รหัสวิชานี้มีอยู่แล้ว	ฐานข้อมูลปฏิเสธเพราะรหัสซ้ำ
413	ไฟล์ใหญ่เกินที่ระบบรองรับ	แก้ค่า max_upload_mb หรือย่อไฟล์
415	ไฟล์ไม่ใช่ PDF, PNG, JPG, TIFF	เลือกไฟล์ใหม่
422	ข้อมูลที่ส่งมาผิดรูปแบบ	หรือโมเดลสร้าง SQL หรือ JSON ที่ใช้ไม่ได้
503	ระบบยังไม่พร้อม	ไม่พบไฟล์ฐานข้อมูล หรือติดต่อ Ollama ไม่ได้

ลำดับการไล่ปัญหาที่เร็วที่สุด

- 1 เปิด /api/health ก่อน ถ้าขึ้น degraded แปลว่ายังไม่ต้องไปดูที่อื่น
- 2 ดู terminal ที่รัน uvicorn ข้อความผิดพลาดเต็ม ๆ อยู่ตรงนั้น
- 3 ถ้าเป็น 503 ให้เช็ค ollama serve ยังรันอยู่ และ db_path ใน .env ซ้ำถูกที่

/docs เพื่อดู api ทั้งหมดของเรา



default

GET / Read Root

Parameters Cancel

No parameters

Servers

These operation-level options override the global server options.

/

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/
```

Server response

Code	Details
200	Response body <pre>{ "Hello": "World" }</pre> Download

XAMPP Control Panel v3.3.0 [Compiled: Apr 6th 2021]

XAMPP Control Panel v3.3.0

Service	Module	PID(s)	Port(s)	Actions
☐	Apache	6004 10920	80, 443	Stop Admin Config Logs
☐	MySQL	8336	3306	Stop Admin Config Logs
☐	FileZilla			Start Admin Config Logs
☐	Mercury			Start Admin Config Logs
☐	Tomcat			Start Admin Config Logs

9:38:31 AM [main] All prerequisites found
9:38:31 AM [main] Initializing Modules
9:38:31 AM [main] Starting Check-Timer
9:38:31 AM [main] Control Panel Ready
12:36:28 PM [Apache] Attempting to start Apache app...
12:36:28 PM [Apache] Status change detected: running
12:36:29 PM [mysql] Attempting to start MySQL app...

index.php?route=/server/databases

Server: 127.0.0.1

Databases SQL

Databases

Create database

ISD_Project

Check all

Drop

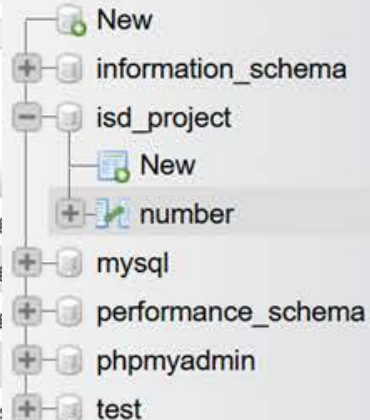
Database Collation

☐	information_schema	utf8_
☐	mysql	utf8mb4_
☐	performance_schema	utf8_
☐	phpmyadmin	
☐	test	latin1_

Total: 5

phpMyAdmin

Recent Favorites



Server: 127.0.0.1 » Database: isd_project » Table: number

Browse

Structure

SQL

Search

Insert

Export

Import

Privileges

Table structure

Relation view

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
☐ 1	ID	int(11)			No	None		AUTO_INCREMENT
☐ 2	DIGIT	int(11)			No	None		
☐ 3	DESCRIPTION	varchar(255)	utf8mb4_general_ci		No	None		
☐ 4	created_at	timestamp			No	current_timestamp()		
☐ 5	updated_at	timestamp			No	current_timestamp()		ON UPDATE CURRENT_TIMESTAMP
☐ 6	deleted_at	timestamp			Yes	NULL		



Check all

With selected:

Browse

Change

Drop

Primary

Unique

Index

Fulltext

Add to central columns

Remove from central columns

Connect api with DB (xampp)

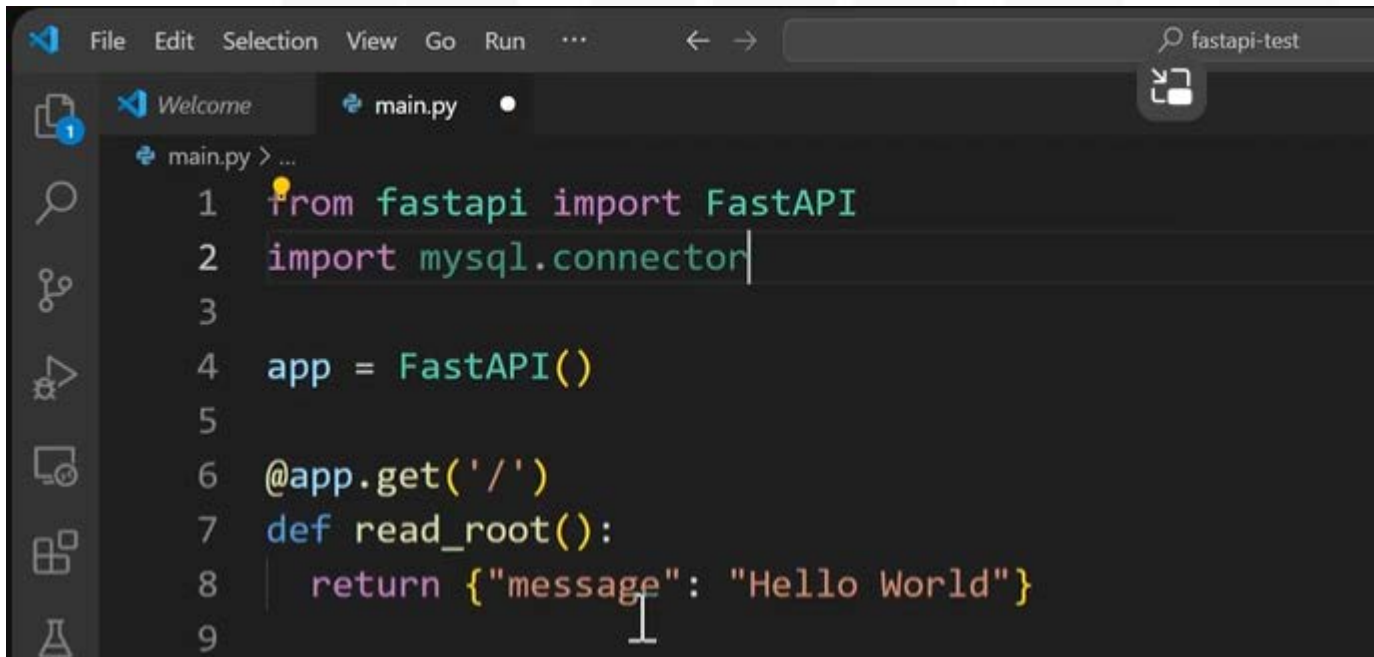
```
INFO: Finished server process [15784]  
INFO: Stopping reloader process [27672]  
PS D:\Teach\Term1\06026240 INTELLIGENT SYSTEM DEVELOPMENT\Lecture\MLOps\handwritten-digit-recognition> pip install mysql-connector-python  
Collecting mysql-connector-python  
  Downloading mysql_connector_python-9.0.0-cp311-cp311-win_amd64.whl.metadata (2.0 kB)  
  Downloading mysql_connector_python-9.0.0-cp311-cp311-win_amd64.whl (14.3 MB)  
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 14.3/14.3 MB 18.0 MB/s eta 0:00:00  
Installing collected packages: mysql-connector-python  
Successfully installed mysql-connector-python-9.0.0
```

mysql-connector-python 8.4.0

```
pip install mysql-connector-python
```

เริ่มทำการติดต่อรหว่าง python กับฐานข้อมูล
control c เพื่อออกจาก INFO แล้ว ติดตั้งโดยใช้คำสั่ง `pip install mysql-connector-python`

ใช้ mysql.connector ในการเชื่อมต่อ DB



```
File Edit Selection View Go Run ... fastapi-test
Welcome main.py
main.py > ...
1 from fastapi import FastAPI
2 import mysql.connector
3
4 app = FastAPI()
5
6 @app.get('/')
7 def read_root():
8     return {"message": "Hello World"}
9
```

```
main.py X {} launch.json  
handwritten-digit-recognition > main.py > ...  
1  from fastapi import FastAPI  
2  
3  # import mysql.connector to connect to the database  
4  import mysql.connector  
5  # create function to connect to the database  
6  def get_db_connection() -> PooledMySQLConnection | MySQLConnection...:  
7      connect_isd_db: PooledMySQLConnection | MyS... = mysql.connector.connect(  
8          host="localhost",  
9          user = "root",  
10         database = "isd_project"  
11     )  
12     return connect_isd_db  
13
```

สร้าง function สำหรับไว้ติดต่อกับ DB
จากนั้นสร้างการเชื่อมต่อกับ DB

main.py • {} launch.json

handwritten-digit-recognition > main.py > ...

14

15 `app = FastAPI()`

```
# create a function to get all the digit data from the database
@app.get("/get_digit_data")
def get_digit_data() -> list[Any]:
    # connect to the database
    db_connection: PooledMySQLConnection | MyS... = get_db_connection()
    # create a cursor object to execute the query
    cursor: MySQLCursorAbstract | Any = db_connection.cursor()
    # execute the query
    cursor.execute("SELECT * FROM number")
    # fetch all the data from the database
    digit_data: List[Tuple[Decimal | bytes ... = cursor.fetchall()
    # close the cursor
    cursor.close()
    # close the database connection
    db_connection.close()
    digits: list[Any] = []
    for row in digit_data:
        digits.append({
            "id": row[0],
            "digit": row[1],
            "meaning": row[2]})
    return digits
```

```
1  [
2    {
3      "id": 1,
4      "digit": 0,
5      "meaning": "Zero - additive identity"
6    },
7    {
8      "id": 2,
9      "digit": 1,
10     "meaning": "One - multiplicative identity"
11   },
12   {
13     "id": 3,
14     "digit": 2,
15     "meaning": "Two - only even prime"
16   },
17   {
18     "id": 4,
19     "digit": 3,
20     "meaning": "Three - first odd prime"
21   },
22   {
23     "id": 5,
24     "digit": 4,
25     "meaning": "Four - equals 2^2 the first square number"
26   },
27   {
28     "id": 6,
29     "digit": 5,
30     "meaning": "Five - second odd prime"
31   },
32   {
33     "id": 7,
34     "digit": 6,
35     "meaning": "Six - equals 1 x 2 x 3 and 1 + 2 + 3 so it is the first perfect number"
36   },
37   {
38     "id": 8,
39     "digit": 7,
40     "meaning": "Seven - third odd prime"
41   },
42   {
43     "id": 9,
44     "digit": 8,
45     "meaning": "Eight - equals 2^3 the first cube"
46   },
47   {
48     "id": 10,
49     "digit": 9,
50     "meaning": "Nine - equals 3^2 the first odd square"
51   }
52 ]
```

For insert data into the database



handwritten-digit-recognition > main.py > insert_digit_data

```
1  from fastapi import FastAPI
2  # import mysql.connector to connect to the database
3  import mysql.connector
4  # import BaseModel from pydantic to insert data into the database
5  from pydantic import BaseModel
6
```

handwritten-digit-recognition > main.py > get_db_connection

```
16  app = FastAPI()
17
18  class Digit(BaseModel):
19      digit: int
20      meaning: str
21  # create api to insert data into the database
22  @app.post("/insert_digit_data")
23  def insert_digit_data(digit: Digit) -> dict[str, Any]:
24      # connect to the database
25      db_connection: PooledMySQLConnection | MyS... = get_db_connection()
26      # create a cursor object to execute the query
27      cursor: MySQLCursorAbstract | Any = db_connection.cursor()
28      # execute the query
29      cursor.execute("INSERT INTO number (digit, description) VALUES (%s, %s)", (digit.digit, digit.meaning))
30      # commit the query
```

127.0.0.1:8000/docs#/default/insert_di...

🔍

☆

⚙️

|

📄

☆

FastAPI

0.1.0

OAS 3.1

/openapi.json

default

POST

/insert_digit_data

Insert Digit Data

Parameters

Cancel

No parameters

Request body

required

application/json

```
{  "digit": 10,  "meaning": "Ten - Ten fight ten"}  
```

Servers

These operation-level options override the global server options.

/

Execute

Clear

Execute

Responses

Curl

```
curl -X 'POST' \  'http://127.0.0.1:8000/insert_digit_data' \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -d '{  "digit": 10,  "meaning": "Ten - Ten fight ten"}'
```

Request URL

http://127.0.0.1:8000/insert_digit_data

Server response

Code	Details
200	Response body <pre>{ "id": 11, "message": "Data inserted successfully"} </pre> Response headers <pre>content-length: 48 content-type: application/json date: Mon, 30 Sep 2024 08:02:59 GMT server: uvicorn</pre>

Responses

Code	Description
200	Successful Response

FASTAPI

phpMyAdmin

Server: 127.0.0.1 » Database: isd_project » Table: number

Browse Structure SQL Search

Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy

Showing rows 0 - 9 (10 total, Query took 0.0002 seconds)

`SELECT * FROM `number``

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code]

Show all | Number of rows: 25 | Filter rows

Extra options

ID	DIGIT	DESCRIPTION
1	0	Zero - additive identity
2	1	One - multiplicative identity
3	2	Two - only even prime
4	3	Three - first odd prime
5	4	Four - equals 2^2 the first square number
6	5	Five - second odd prime
7	6	Six - equals 1 x 2 x 3 and 1 + 2 + 3 so it is the ..
8	7	Seven - third odd prime
9	8	Eight - equals 2^3 the first cube
10	9	Nine - equals 3^2 the first odd square

localhost/phpmyadmin/index.php?route=/sql&db=isd_project&table=number

Server: 127.0.0.1 » Database: isd_project » Table: number

Browse Structure SQL Search Insert Export

Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy

Showing rows 0 - 10 (11 total, Query took 0.0003 seconds.)

`SELECT * FROM `number``

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

ID	DIGIT	DESCRIPTION	created_at	updated_at
1	0	Zero - additive identity	2024-09-30 14:11:09	2024-09-30 14:11:09
2	1	One - multiplicative identity	2024-09-30 14:11:56	2024-09-30 14:11:56
3	2	Two - only even prime	2024-09-30 14:15:21	2024-09-30 14:15:21
4	3	Three - first odd prime	2024-09-30 14:15:53	2024-09-30 14:15:53
5	4	Four - equals 2^2 the first square number	2024-09-30 14:16:37	2024-09-30 14:16:37
6	5	Five - second odd prime	2024-09-30 14:16:59	2024-09-30 14:16:59
7	6	Six - equals 1 x 2 x 3 and 1 + 2 + 3 so it is the ...	2024-09-30 14:17:30	2024-09-30 14:17:30
8	7	Seven - third odd prime	2024-09-30 14:17:51	2024-09-30 14:17:51
9	8	Eight - equals 2^3 the first cube	2024-09-30 14:18:23	2024-09-30 14:18:23
10	9	Nine - equals 3^2 the first odd square	2024-09-30 14:19:12	2024-09-30 14:19:12
11	10	Ten - Ten fight ten	2024-09-30 15:03:00	2024-09-30 15:03:00

หลังจากใช้ METHOD POST เพื่อเพิ่ม DIGIT 10

Step 3: สร้างไฟล์ .env — เก็บ Config ทั้งหมด

สร้างไฟล์ชื่อ .env ที่ root ของโปรเจก (ห้าม commit ขึ้น Git!)

--- Database Config --- (ทุกกลุ่มที่มีฐานข้อมูล)

DATABASE_URL=sqlite:///./mydb.db # ค่าเริ่มต้นของแอป ไม่ต้องติดตั้ง DB

DATABASE_URL=mysql+mysqlconnector://root:@localhost:3306/mydb # ถ้าใช้ XAMPP

DB_ECHO=False # True = แสดง SQL log ใน console

--- Model Config --- (เฉพาะกลุ่มที่เรียกโมเดล ใส่เท่าที่ใช้จริง)

MODEL_NAME=qwen2.5:7b # ชื่อโมเดลที่เรียก เช่น Qwen บน Ollama

MODEL_BASE_URL=http://localhost:11434/v1 # endpoint ของโมเดล

MODEL_API_KEY=xxxxxxx # ถ้าเรียกโมเดลบน cloud (รันเครื่องตัวเองไม่ต้องใส่)

--- App Config ---

APP_HOST=0.0.0.0 APP_PORT=8000 DEBUG=True

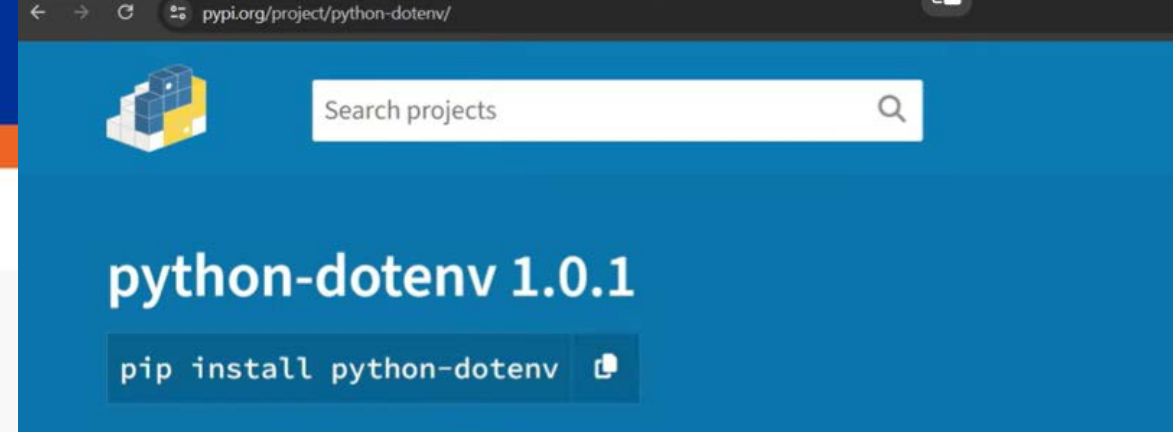
เพิ่ม .env ใน .gitignore: echo ".env" >> .gitignore

แยก database connection file

สร้างไฟล์ .env

ไปเพิ่มการเรียก env ที่ main.py บรรทัดที่ 6-8

จากนั้นแก้ไขส่วนที่เรียกการเชื่อมต่อ db ใน function get_db_connection ที่เรา fix ไว้ในตอนแรก



handwritten-digit-recognition > main.py > get_db_connection

```
5 from pydantic import BaseModel
6 # import .env file
7 from dotenv import load_dotenv
8 import os
9 load_dotenv()
10
11 # create function to connect to the database
12 def get_db_connection() -> PooledMySQLConnection | My
13     connect_isd_db: PooledMySQLConnection | MyS... =
14         # host="localhost",
15         host = os.getenv("DB_HOST"),
16         user = os.getenv("DB_USER"),
17         database = os.getenv("DB_NAME"),
18     )
19     return connect_isd_db
```

handwritten-digit-recognition > .env

```
1 DB_HOST = localhost
2 DB_USER = root
3 DB_NAME = isd_project
```